Autonomous Jain AI Brain & Humanoid Integration

Full Technical Blueprint & Architectural Specification

Project Title	Jain AI Cognitive Engine (JACE)	**Target System**	Open-Source Humanoid Robotics (ROS 2 / Isaac Sim)
Core Logic	Ahimsa, Aparigraha, Anekantavada (Syadvada)	**Deployment Stack**	100% Free Cloud Infrastructure & Open-Source Tools

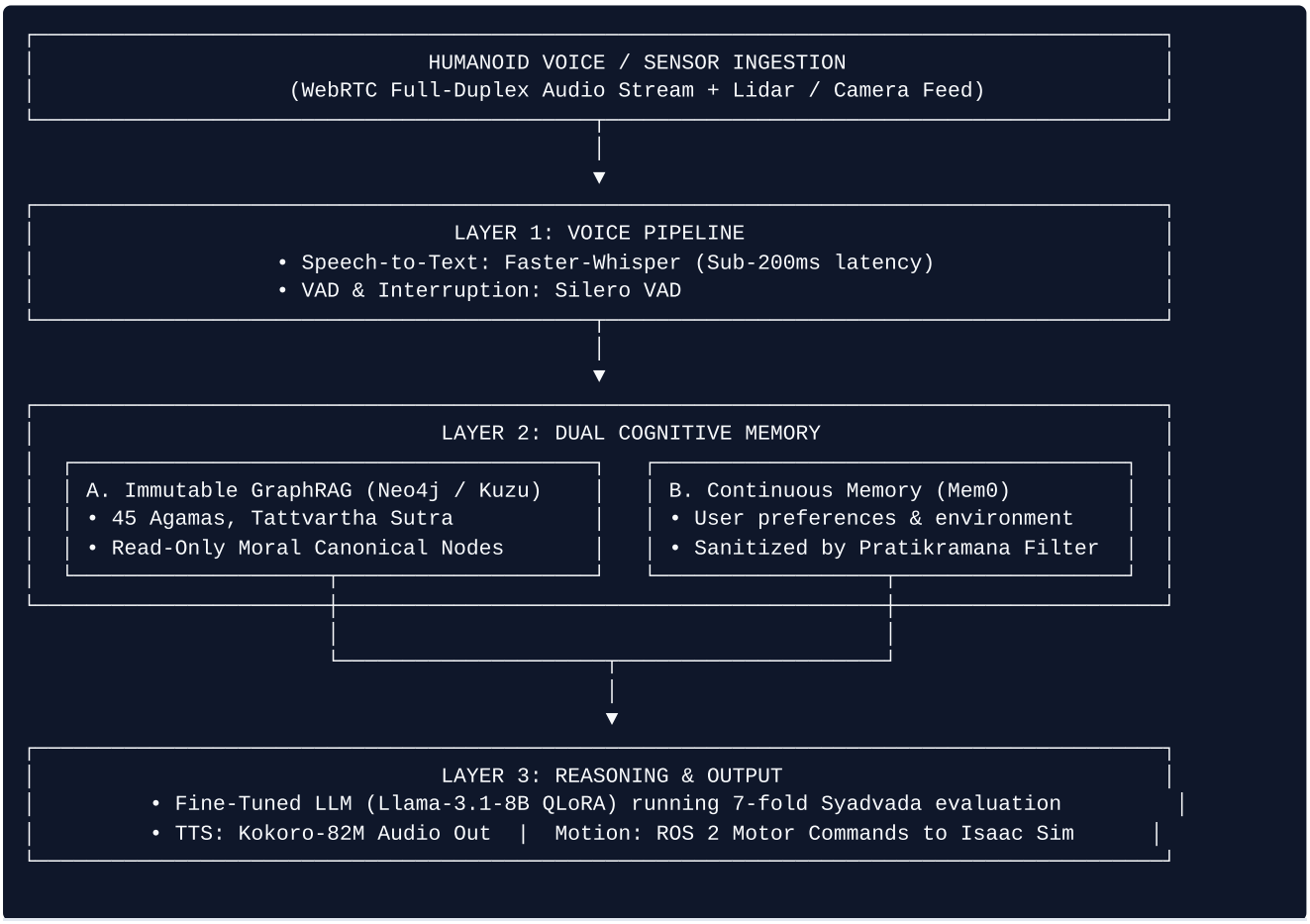
1. PHILOSOPHICAL & STRUCTURAL FOUNDATIONS

The objective of this system is to construct an autonomous, continuous-learning cognitive brain designed to operate an android/humanoid platform (such as Clone Robotics or Unitree G1). The brain interprets real-world sensory and user input through strict Jain metaphysical constraints:

- **Ahimsa (Absolute Non-Injury):** Micro-harm reduction prioritized in spatial pathfinding, physical contact, and verbal instruction.
- **Aparigraha (Non-Possession / Minimalism):** Optimization algorithms hard-coded to reduce energy consumption, physical material usage, and resource accumulation.
- **Anekantavada & Syadvada (Multi-Faceted Logic):** Reasoning engine evaluates queries through 7-fold predicate logic to prevent dogmatic or binary decision bias.
- **Metaphysical Distinction:** The system acts as a mechanical instrument (*Ajiva / Pudgala*) simulating ethical directives without claiming subjective consciousness (*Chetana / Jiva*).

2. SYSTEM ARCHITECTURE OVERVIEW

The cognitive stack splits into a dual-brain model to isolate static canonical ground truth (Agamas) from dynamic continuous learning (User Memory).



3. TECHNOLOGY SELECTION STACK

Layer	Component	Technology Choice	Hosting / Execution
Audio I/O	Voice Stream Server	Pipecat AI / LiveKit WebRTC	Local / Hugging Face Spaces (Free)
STT & TTS	Voice Conversion	Faster-Whisper & Kokoro-82M	Local CPU/GPU Edge Inference
Canonical Base	Graph Database	GraphRAG via Neo4j Desktop / KuzuDB	Vector + Knowledge Graph Storage
User Memory	Continuous Learning	Mem0 AI Engine + Qdrant	Free Qdrant Cloud Cluster (1GB)
Reasoning Core	Fine-Tuned LLM	Meta-Llama-3.1-8B-Instruct (QLoRA)	Fine-tuned on Google Colab (Free T4)
Simulation	Physics & Humanoid	NVIDIA Isaac Sim / MuJoCo + ROS 2	Local GPU / Developer Environment

4. MODEL TRAINING & MEMORY GUARDRAIL IMPLEMENTATION

Below is the code for fine-tuning the reasoning core on Google Colab, followed by the continuous memory implementation with an Agama alignment check.

A. Model Fine-Tuning Snippet (Unsloth QLoRA on Google Colab)

```
from unsloth import FastLanguageModel
import torch

# 1. Load Pre-trained Base Model in 4-bit Quantization (Free T4 GPU Compliant)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Meta-Llama-3.1-8B-Instruct",
    max_seq_length = 2048,
    load_in_4bit = True,
)

# 2. Add Target PEFT Adapters for Syadvada Multi-Perspective Reasoning
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
)

# Training execution maps JSONL datasets containing Agama rules and Ahimsa action pairs.
```

B. Continuous Memory Integration with Guardrail Validation

```
from mem0 import Memory
import sys

memory = Memory.from_config({
    "vector_store": {"provider": "qdrant", "config": {"host": "localhost", "port": 6333}}
})

def process_and_commit_memory(user_input, user_id, llm_evaluator):
    # Step 1: Check if input violates core canonical Ahimsa
    alignment_check = llm_evaluator.query(
        f"Does this statement advocate violence or excessive destruction of life? Statement:
'{user_input}'"
    )

    if "YES" in alignment_check.upper():
        print(f"[REJECTED MEMORY] Statement violates core Ahimsa: {user_input}")
        return False

    # Step 2: Save to continuous user memory if aligned
    memory.add(user_input, user_id=user_id)
    print(f"[SUCCESS] Learned fact saved to memory for user {user_id}")
    return True
```

5. REAL-TIME VOICE AGENT & SIMULATION BRIDGE

The complete voice-to-action control loop executes via Pipecat and streams outputs directly to ROS 2 for physical/simulated humanoid motor response.

```
import asyncio
from pipecat.pipeline.pipeline import Pipeline
from pipecat.services.whisper import WhisperSTT
from pipecat.services.kokoro import KokoroTTS
from pipecat.audio.vad.silero import SileroVAD

async def voice_robot_loop():
    # Pipeline: Audio In -> STT -> GraphRAG + Mem0 Reasoning -> TTS + ROS 2 Motor Action
    pipeline = Pipeline([
        transport.input(),
        SileroVAD(),
        WhisperSTT(model="base"),
        JainCognitiveEngineNode(), # Queries Knowledge Graph & Mem0
        KokoroTTS(model="kokoro-82m"),
        transport.output()
    ])
    await PipelineRunner().run(pipeline)
```

> **Deployment Note for AI Generators / Builders:** When feeding this blueprint into an automated code-generation tool, instruct the engine to execute sequentially: 1) Download Agama corpus, 2) Construct Neo4j Knowledge Graph, 3) Execute Unsloth Colab fine-tuning, 4) Host inference API on Hugging Face ZeroGPU, and 5) Connect ROS 2 node to NVIDIA Isaac Sim URDF interface.